Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

Operating Systems

Laboratory Work No. 5

The File System with Your Own Hands

Track A: Warm-up on Linux

Elaborated by:

st. gr. FAF-241
Luchiciov Alexei

Verified by:

Reiţman Patricia

Chișinău – 2026

Contents

1. Introduction....3
2. The Tree and File Types....3
  2.1. Observations....4
  2.2. Interesting Command Output....4
3. Inodes and Links....4
  3.1. Observations....5
  3.2. Interesting Command Output....5
4. Permissions....6
  4.1. Observations....6
  4.2. Interesting Command Output....7
5. Disks and the Kernel as Files....7
  5.1. Observations....8
  5.2. Interesting Command Output....8
6. Conclusion....8

1. Introduction

The purpose of this laboratory work was to explore the Linux file system directly from the terminal. The experiment covered the directory tree and file types, inodes and links, permissions, special permission bits, block devices, mounted file systems, and the /proc pseudo-file system.

The work was performed in Ubuntu running inside the Oracle VirtualBox virtual machine. All commands were executed as the normal user alex_007m, because using root would bypass many permission checks and would make several experiments misleading.

2. The Tree and File Types

First, I inspected the root of the Linux file-system tree and several paths with different file types:

```
$ ls /
bin    dev   lib         media  proc  sbin  sys  var
boot   etc   lib64       mnt    root  snap  tmp
cdrom  home  lost+found  opt    run   srv   usr

$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root     7 Apr 20 08:46 /bin -> usr/bin
crw-rw-rw-   1 root root  1, 3 Oct 12 12:50 /dev/null
drwxr-xr-x 141 root root 12288 Oct 12 10:46 /etc
-rw-r--r--   1 root root  2826 Oct  5 12:58 /etc/passwd
drwxrwxrwt  15 root root   360 Oct 12 12:52 /tmp
```

The file command and readlink were used to identify regular files and symbolic links:

```
$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: symbolic link to ../lib/cargo/bin/coreutils/ls
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin

$ readlink -f /bin
/usr/bin
```

I also compared the size of two system directories and searched /dev for character and block devices:

```
$ du -sh /var/log 2>/dev/null
140M    /var/log
$ du -sh /usr 2>/dev/null
5.0G    /usr

$ find /dev -maxdepth 1 -type c | head
/dev/fb0
/dev/i2c-0
/dev/zfs
...

$ find /dev -maxdepth 1 -type b | head
/dev/loop16
/dev/loop15
...
/dev/sda2
```

2.1. Observations

1. The first character in ls -l identifies the file type. /etc and /tmp start with d, so they are directories. /etc/passwd starts with -, so it is a regular file. /bin starts with l, so it is a symbolic link. /dev/null starts with c, so it is a character device.

2. On this system, /bin is not a separate directory. It is a symbolic link to usr/bin, and readlink -f /bin resolves it to /usr/bin.

3. Examples of character devices are /dev/fb0 and /dev/null. Examples of block devices are /dev/loop16 and /dev/sda2.

4. /usr is much larger than /var/log: about 5.0 GB compared with 140 MB. In this VM, /usr is roughly 36 times larger.

2.2. Interesting Command Output

```
lrwxrwxrwx 1 root root 7 Apr 20 08:46 /bin -> usr/bin
crw-rw-rw- 1 root root 1, 3 Oct 12 12:50 /dev/null
```

This output is a compact example of how Linux represents different objects in one file-system namespace. A symbolic link and a character device both appear as paths, but their first permission character immediately shows that they are different object types.

3. Inodes and Links

The next experiment demonstrated that a file name is separate from the inode that stores the file metadata. I created notes.txt and inspected it using stat and ls -li:

```
$ stat notes.txt
File: notes.txt
Size: 6          Blocks: 8          IO Block: 4096   regular file
Device: 8,2      Inode: 1575858     Links: 1
Access: (0664/-rw-rw-r--)  Uid: (1000/alex_007m)  Gid: (1000/alex_007m)

$ ls -li notes.txt
1575858 -rw-rw-r-- 1 alex_007m alex_007m 6 Oct 12 12:53 notes.txt
```

A hard link was then created. Both names referred to the same inode and the link count increased to two:

```
$ ln notes.txt hard.txt
$ ls -li notes.txt hard.txt
1575858 -rw-rw-r-- 2 alex_007m alex_007m 6 Oct 12 12:53 hard.txt
1575858 -rw-rw-r-- 2 alex_007m alex_007m 6 Oct 12 12:53 notes.txt

$ rm notes.txt
$ cat hard.txt
hello
$ ls -li hard.txt
1575858 -rw-rw-r-- 1 alex_007m alex_007m 6 Oct 12 12:53 hard.txt
```

I then created a symbolic link. Unlike the hard link, the symbolic link had its own inode and stored the target path as text:

```
$ ln -s hard.txt soft.txt
$ ls -li hard.txt soft.txt
1575858 -rw-rw-r-- 1 alex_007m alex_007m 6 Oct 12 12:53 hard.txt
1575859 lrwxrwxrwx 1 alex_007m alex_007m 8 Oct 12 12:54 soft.txt -> hard.txt
```

```
$ cat soft.txt
hello
$ rm hard.txt
$ cat soft.txt
cat: soft.txt: No such file or directory
$ ls -l soft.txt
lrwxrwxrwx 1 alex_007m alex_007m 8 Oct 12 12:54 soft.txt -> hard.txt
```

Directory link counts were also observed:

```
$ mkdir d ; ls -ld d
drwxrwxr-x 2 alex_007m alex_007m 4096 Oct 12 12:54 d

$ mkdir d/x d/y ; ls -ld d
drwxrwxr-x 4 alex_007m alex_007m 4096 Oct 12 12:54 d
```

Finally, I compared logical size with allocated disk space:

```
$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-rw-r-- 1 alex_007m alex_007m 2 Oct 12 12:54 one.txt
4.0K    one.txt

$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-rw-r-- 1 alex_007m alex_007m 1.0G Oct 12 12:55 big.img
0    big.img
```

3.1. Observations

1. In stat, the inode number is shown after “Inode:” and the number of names referring to the inode is shown after “Links:”. For notes.txt the inode was 1575858 and the initial link count was 1.

2. After rm notes.txt, hard.txt could still be read because both names referred to the same inode. rm removes a directory entry (a name). The file data remains while at least one hard link still refers to the inode and the file is not otherwise held open.

3. soft.txt failed after hard.txt was removed because a symbolic link stores the target path, not the target inode. soft.txt still existed, but its target name no longer existed. Its size was 8 bytes because the stored target string “hard.txt” contains exactly 8 characters.

4. A new directory has a link count of 2: one link is its own “.” entry and one is its name in the parent directory. Each immediate subdirectory contains a “..” entry pointing back to its parent, so creating d/x and d/y increased d's link count from 2 to 4.

5. one.txt contains only two bytes, but the ext4 file system allocates disk space in blocks, so du reported 4.0K. The 1 GB big.img created with truncate was a sparse file: it had a large logical size but no data blocks had been allocated, so du reported 0.

3.2. Interesting Command Output

```
1575858 -rw-rw-r-- 2 alex_007m alex_007m 6 ... hard.txt
1575858 -rw-rw-r-- 2 alex_007m alex_007m 6 ... notes.txt

1575859 lrwxrwxrwx 1 alex_007m alex_007m 8 ... soft.txt -> hard.txt
```

The identical inode number for the two hard-link names and the different inode for the symbolic link made the difference between the two link types visible directly.

4. Permissions

I first changed permissions on a regular file and verified that Linux enforced them even though I owned the file:

```
$ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 alex_007m alex_007m 7 Oct 12 12:55 s.txt

$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied

$ chmod u+r s.txt ; cat s.txt
secret
```

The current umask and default permissions for newly created files and directories were:

```
$ umask
0002

$ touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
-rw-rw-r-- 1 alex_007m alex_007m    0 Oct 12 12:55 new.txt
drwxrwxr-x 2 alex_007m alex_007m 4096 Oct 12 12:55 newdir
```

The directory experiment showed that read and execute mean different things for directories:

```
$ chmod u=r,go= box ; ls box ; cat box/f
f
cat: box/f: Permission denied

$ chmod u=x,go= box ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi
```

Finally, I inspected the setuid bit on passwd, the sticky bit on /tmp, and the protected /etc/shadow file:

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 93640 Feb  2  2026 /usr/bin/passwd

$ ls -ld /tmp
drwxrwxrwt 16 root root 380 Oct 12 12:55 /tmp

$ cat /etc/shadow
cat: /etc/shadow: Permission denied
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1337 Oct 5 12:58 /etc/shadow
```

4.1. Observations

1. The numeric modes correspond to: 600 = rw-------, 644 = rw-r--r--, 755 = rwxr-xr-x, and 700 = rwx------. A private key or secret file can use 600, a normal readable text file can use 644, an executable program or public directory can use 755, and a private directory can use 700.

2. Ownership identifies which permission class applies, but it does not automatically grant access. With chmod 000, the owner permission bits are also empty, so the owner is not allowed to read the file.

3. The umask was 0002. New regular files start from a maximum of 666, so 666 minus the masked write bit for “others” becomes 664 (rw-rw-r--). New directories start from 777, so the same umask produces 775 (rwxrwxr-x).

4. For a directory, r allows listing the names stored in the directory, while x allows traversing the directory and accessing an entry when its name is known. Therefore read-only box could be listed but box/f could not be opened; execute-only box could not be listed, but cat box/f succeeded because the name f was already known.

5. The s in -rwsr-xr-x is the setuid bit. /usr/bin/passwd runs with the effective user ID of its owner, root, so a normal user can securely update protected password information. The t in drwxrwxrwt is the sticky bit: in /tmp, users may create files, but normally may delete or rename only files they own (or files managed by the directory owner/root).

4.2. Interesting Command Output

```
chmod u=r,go= box:
ls box      -> f
cat box/f   -> Permission denied

chmod u=x,go= box:
ls box      -> Permission denied
cat box/f   -> hi
```

This was the clearest permission experiment because it showed that directory read permission and directory execute permission solve different problems: listing names versus traversing the directory.

5. Disks and the Kernel as Files

The final part examined the block devices visible to the virtual machine and the root file system:

```
$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda      8:0    0    30G  0 disk
|-sda1   8:1    0     1M  0 part
`-sda2   8:2    0    30G  0 part /
sr0     11:0    1  50.7M  1 rom  /run/media/alex_007m/VBox_GAs_7.2.20

$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        30G  7.6G   21G  28% /

$ mount | grep " / "
/dev/sda2 on / type ext4 (rw,relatime)
```

The /proc pseudo-file system was then used to inspect CPU, memory, process status, and file descriptors:

```
$ grep -c processor /proc/cpuinfo
4

$ head -3 /proc/meminfo
MemTotal:        3479664 kB
MemFree:          627728 kB
MemAvailable:    2211600 kB
```

```
$ head -5 /proc/self/status
Name:    head
Umask:   0002
State:   R (running)
Tgid:    4603
Ngid:    0
```

```
$ ls -l /proc/self/fd
total 0
lrwx------ 1 alex_007m alex_007m 64 ... 0 -> /dev/pts/1
lrwx------ 1 alex_007m alex_007m 64 ... 1 -> /dev/pts/1
lrwx------ 1 alex_007m alex_007m 64 ... 2 -> /dev/pts/1
lr-x------ 1 alex_007m alex_007m 64 ... 3 -> /proc/4606/fd
```

5.1. Observations

1. The VM has a 30 GB virtual disk named sda. It contains a tiny sda1 partition and a 30 GB sda2 partition. The root directory / is mounted from /dev/sda2, and mount reported the file-system type as ext4.

2. The root file system was 28% full: 7.6 GB used out of 30 GB, with about 21 GB available.

3. /proc/cpuinfo reported 4 processors. /proc/meminfo reported 3,479,664 kB of total RAM, which is about 3.3 GiB.

4. /proc/self/status showed Name: head because /proc/self refers to the process that accesses it. In the command head -5 /proc/self/status, the process opening the path is the head program itself, so “self” resolves to head.

5.2. Interesting Command Output

```
/dev/sda2 on / type ext4 (rw,relatime)

Name:    head
State:   R (running)
```

These two short outputs connect two important ideas from the laboratory: the file-system tree is backed by a real mounted block device, while /proc presents live kernel and process information through file-like paths.

6. Conclusion

In this laboratory work I examined the Linux file system from several levels. I identified file types in the directory tree, saw how names refer to inodes, compared hard links with symbolic links, and observed that deleting a name does not necessarily delete the underlying data immediately.

The permission experiments showed that ownership and permission bits are separate concepts and that directory read and execute permissions have different meanings. The setuid and sticky bits demonstrated how Linux adds special rules for security-sensitive programs and shared directories.

Finally, I connected the file-system tree to the virtual disk and used /proc to read live kernel information about CPU, memory, processes, and file descriptors. The laboratory made the idea “everything is represented through file-like interfaces” much more concrete.